AUDIOWEB: PLATAFORMA DE CONTENIDO INTERACTIVO

Realizado por: Rubén Ruiz Ortiz

Fecha: 12/12/25

Profesor: Teresa Alonso Aguilar

Curso: Base de datos

ÍNDICE

1.- ENUNCIADO DEL PROBLEMA 📄 3
2.- MODELO ENTIDAD/RELACIÓN 4
3.- PASO A MODELO RELACIONAL EN WORKBENCH 5
3.1-CAMBIOS CON SET Y ENUM 6
4.-CARGA MASIVA DE DATOS 7
5.-CONSULTAS SQL DEL PROYECTO 7
CONSULTA Nº1 8
CONSULTA Nº2 9
CONSULTA Nº3 10
CONSULTA Nº4 11
CONSULTA Nº5 12
CONSULTA Nº6 13
CONSULTA Nº7 14
6.FUNCIONES Y PROCEDIMIENTOS DEL PROYECTO 15
Función 1: 15
Función 2: 18
Procedimiento 1: 19
Procedimiento 2: 21
Procedimiento 3: 23
7. TRIGGERS DEL PROYECTO 24
Trigger 1: 24
Trigger 2: 27
8.SUBIDA A GITHUB 32
9.CONCLUSIÓN DEL PROYECTO 33

1.- ENUNCIADO DEL PROBLEMA 📄

La aplicación desarrollada es una plataforma de contenidos audiovisuales orientada a la publicación y visualización de vídeos y a la interacción entre los usuarios. Se ha elegido este tipo de aplicación debido al auge de las plataformas de contenido audiovisual en formato vertical, muy utilizadas actualmente en dispositivos móviles, lo que permite modelar de forma realista las entidades y relaciones propias de este tipo de sistemas.

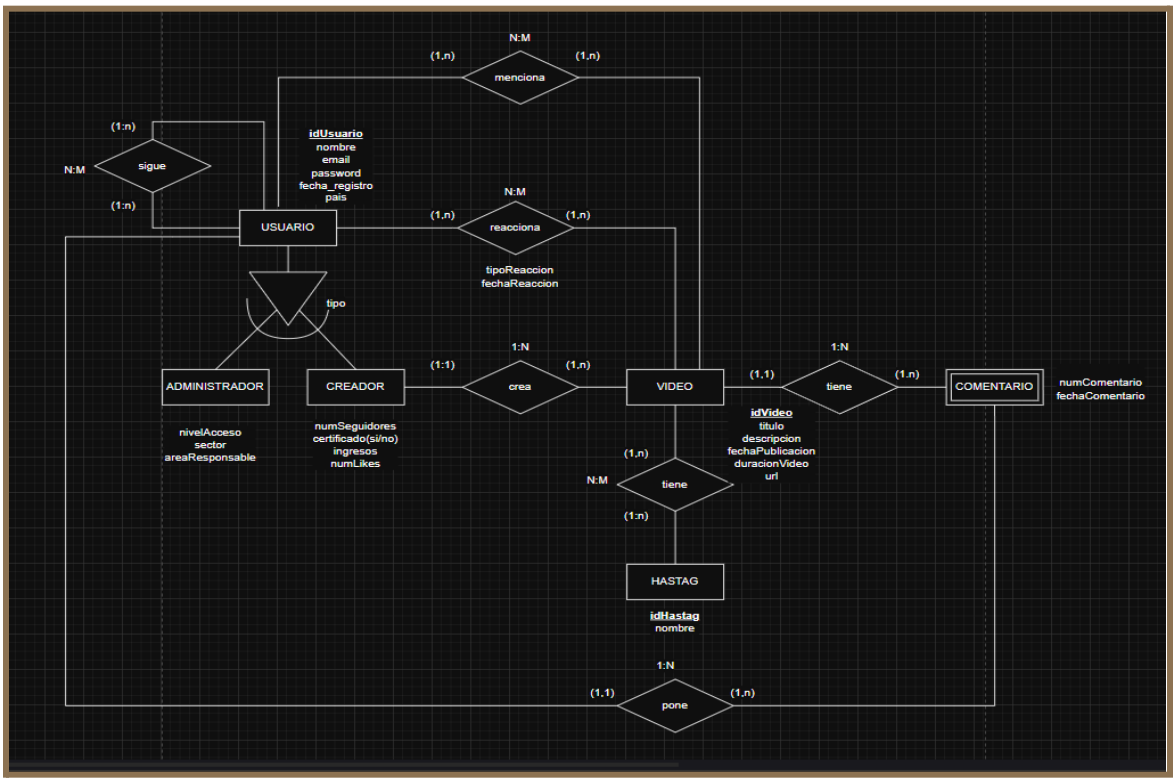
El sistema almacena la siguiente información y dinámicas:

- **Usuarios:** Se registra su identificador único, nombre, correo electrónico, contraseña, fecha de registro y país. Los usuarios pueden seguirse entre sí de forma recíproca (un usuario sigue a varios y es seguido por varios). Se dividen en dos subtipos:
    - *Administradores:* Almacenan su nivel de acceso, sector y área responsable.
    - *Creadores de contenido:* Registran su número de seguidores, estado de certificación, ingresos generados y "me gusta" totales.
- **Vídeos:** Cada creador puede publicar múltiples vídeos, pero cada vídeo pertenece a un único creador. De ellos se guarda un ID único, título, descripción, fecha de publicación, duración y URL.
- **Interacciones (Reacciones y Menciones):** Los usuarios pueden reaccionar a múltiples vídeos (y viceversa), registrando el tipo de reacción y la fecha. Igualmente, los usuarios pueden mencionar distintos vídeos, y un vídeo puede ser mencionado por varios usuarios.
- **Comentarios:** Un usuario puede escribir múltiples comentarios, pero cada comentario es escrito por un único usuario y pertenece a un único vídeo. Se identifican mediante un número correlativo dentro del vídeo y la fecha de publicación.
- **Hashtags:** Identificados por un ID único y nombre. Permiten clasificar el contenido; un vídeo puede tener varios hashtags y un hashtag puede asociarse a distintos vídeos.

2.- MODELO ENTIDAD/RELACIÓN

El diseño de nuestra plataforma comienza con el Modelo Entidad/Relación, donde definimos la estructura de nuestra base de datos y las reglas de interacción entre las distintas entidades.

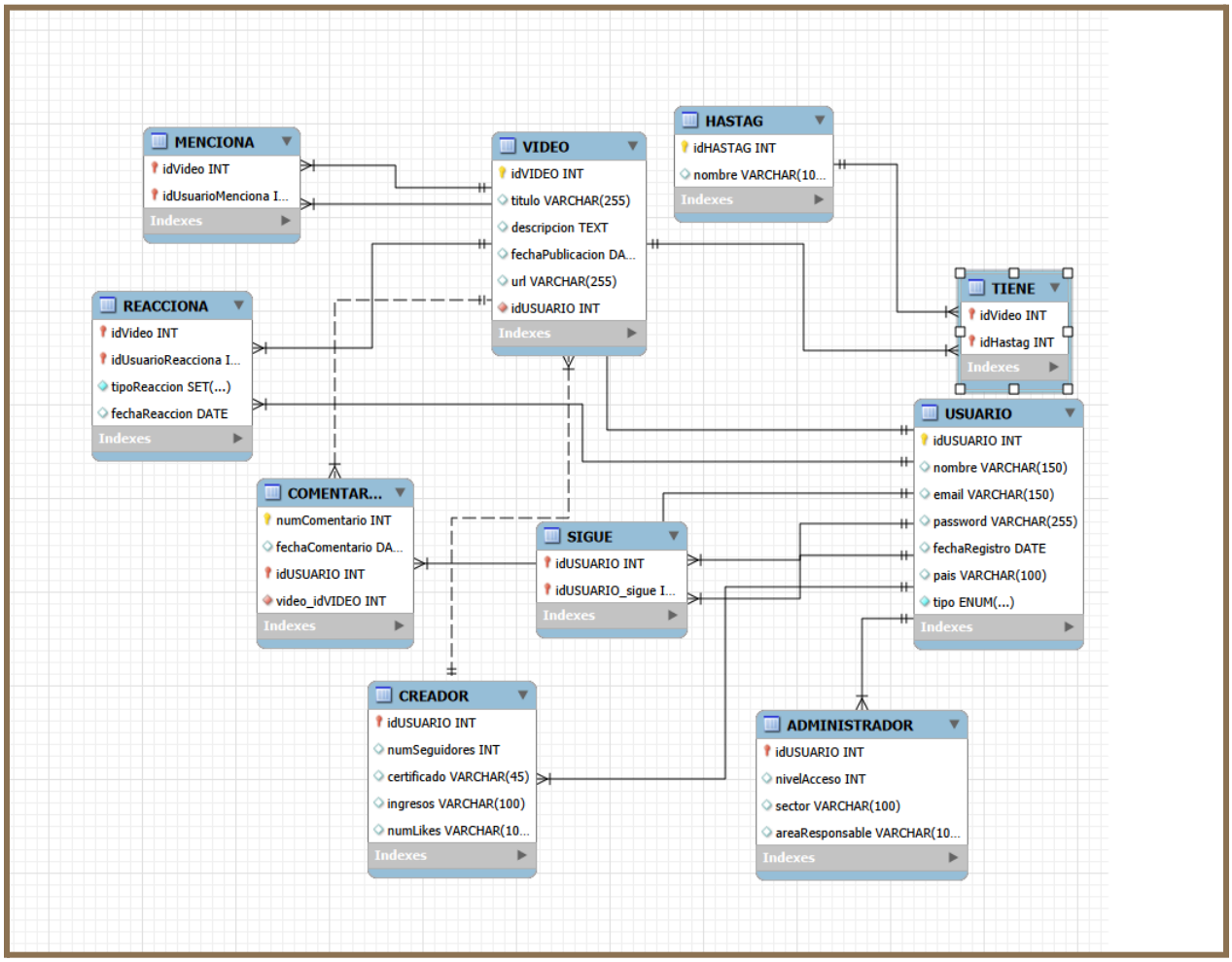
- **Jerarquía de Usuarios:** La entidad principal *Usuario* se divide en *Creadores* (generadores de contenido y gestores de ingresos) y *Administradores* (encargados de la gestión interna).
- **Relaciones principales:** Destaca la "creación" (vincula cada vídeo a un único creador) y el "seguimiento" (relación reflexiva entre usuarios).
- **Interacciones:** Las acciones de "reacción" y "mención" permiten a múltiples usuarios interactuar con diversos vídeos, registrando datos como la fecha y el tipo de respuesta.
- **Entidad Débil (Comentario):** Su existencia depende estrictamente de un vídeo concreto; si este se elimina, el comentario desaparece del sistema.
- **Hashtags:** Sistema de etiquetas para agrupar masivamente los vídeos por temas y optimizar las búsquedas de los usuarios.



3.- PASO A MODELO RELACIONAL EN WORKBENCH

Tras definir el mapa visual del proyecto, utilizamos MySQL Workbench para construir la estructura real de la base de datos, transformando las entidades en tablas físicas donde cada dato se organiza en filas y columnas.

Finalmente, hemos ajustado la configuración de las columnas mediante reglas como ENUM y SET, lo que obliga al sistema a aceptar solo opciones válidas y predefinidas, garantizando que la base de datos esté siempre limpia, ordenada y libre de fallos al introducir la información manualmente



3.1-CAMBIOS CON SET Y ENUM

En concreto, se han aplicado en los siguientes casos:

- **ENUM (Opción única):** En la tabla USUARIO, para restringir que el tipo de cuenta sea estrictamente 'creador' o 'administrador'.
- **SET (Opciones múltiples):** En la tabla REACCIONA, para definir un catálogo cerrado de respuestas permitidas ('Me encanta', 'Like', 'Asombro', 'Divertido', 'No me gusta').

```
alter table USUARIO modify column tipo enum('creador','administrador') not null;


alter table REACCIONA drop column tipoReaccion;

alter table REACCIONA
add column tipoReaccion set('Me encanta', 'Like', 'Asombro', 'Divertido', 'No me gusta')
not null default 'No me gusta' after idUsuarioReacciona;
```

4.-CARGA MASIVA DE DATOS

Para poder realizar mi trabajo, tuve que hacer una carga masiva de datos y así, poder completar las tablas de mi base de datos AudioWeb. Para ello, saqué mi Script de mi modelo en workbench y posteriormente, se lo pasé a la IA para que me lo corrigiera, es decir, me pusiera correctamente los formatos de cada uno de los datos. Finalmente, le pedí los archivos .csv, después de que le ordenará de manera meticulosa lo que debería aparecer en cada tabla y en concreto de cada columna.

5.-CONSULTAS SQL DEL PROYECTO

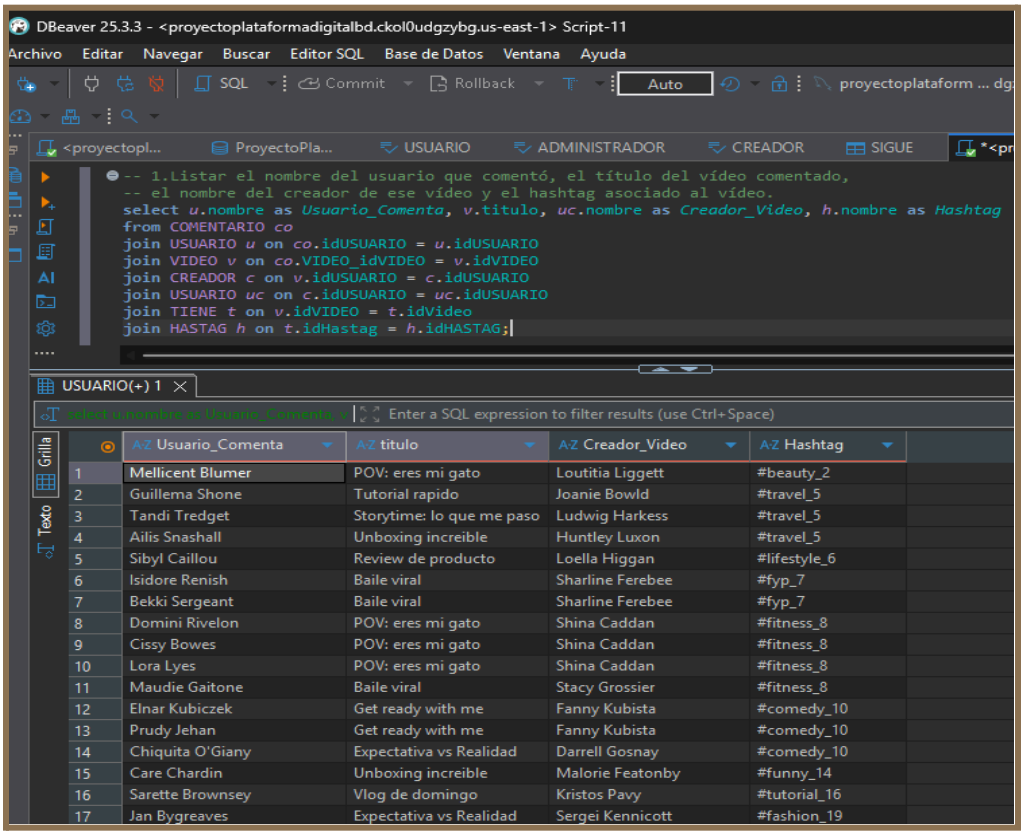
En este apartado es donde realmente vemos para qué sirve todo el trabajo anterior, ya que usamos el lenguaje SQL para hacerle preguntas a la base de datos y obtener respuestas útiles.

En lugar de mirar tablas sueltas, aquí combinamos la información de los usuarios, los vídeos y las interacciones para generar reportes reales, como saber qué hashtags son los más usados o qué creadores están teniendo más éxito.Básicamente, esta sección demuestra cómo transformar un montón de datos guardados en información valiosa para gestionar la plataforma de forma inteligente.

CONSULTA Nº1

Enunciado: Listar el nombre del usuario que comentó, el título del vídeo comentado, el nombre del creador de ese vídeo y el hashtag asociado al vídeo.

```
select u.nombre as Usuario_Comenta, v.titulo, uc.nombre as Creador_Video, h.nombre as
Hashtag
from COMENTARIO co
join USUARIO u on co.idUSUARIO = u.idUSUARIO
join VIDEO v on co.VIDEO_idVIDEO = v.idVIDEO
join CREADOR c on v.idUSUARIO = c.idUSUARIO
join USUARIO uc on c.idUSUARIO = uc.idUSUARIO
join TIENE t on v.idVIDEO = t.idVideo
join HASTAG h on t.idHastag = h.idHASTAG;
```

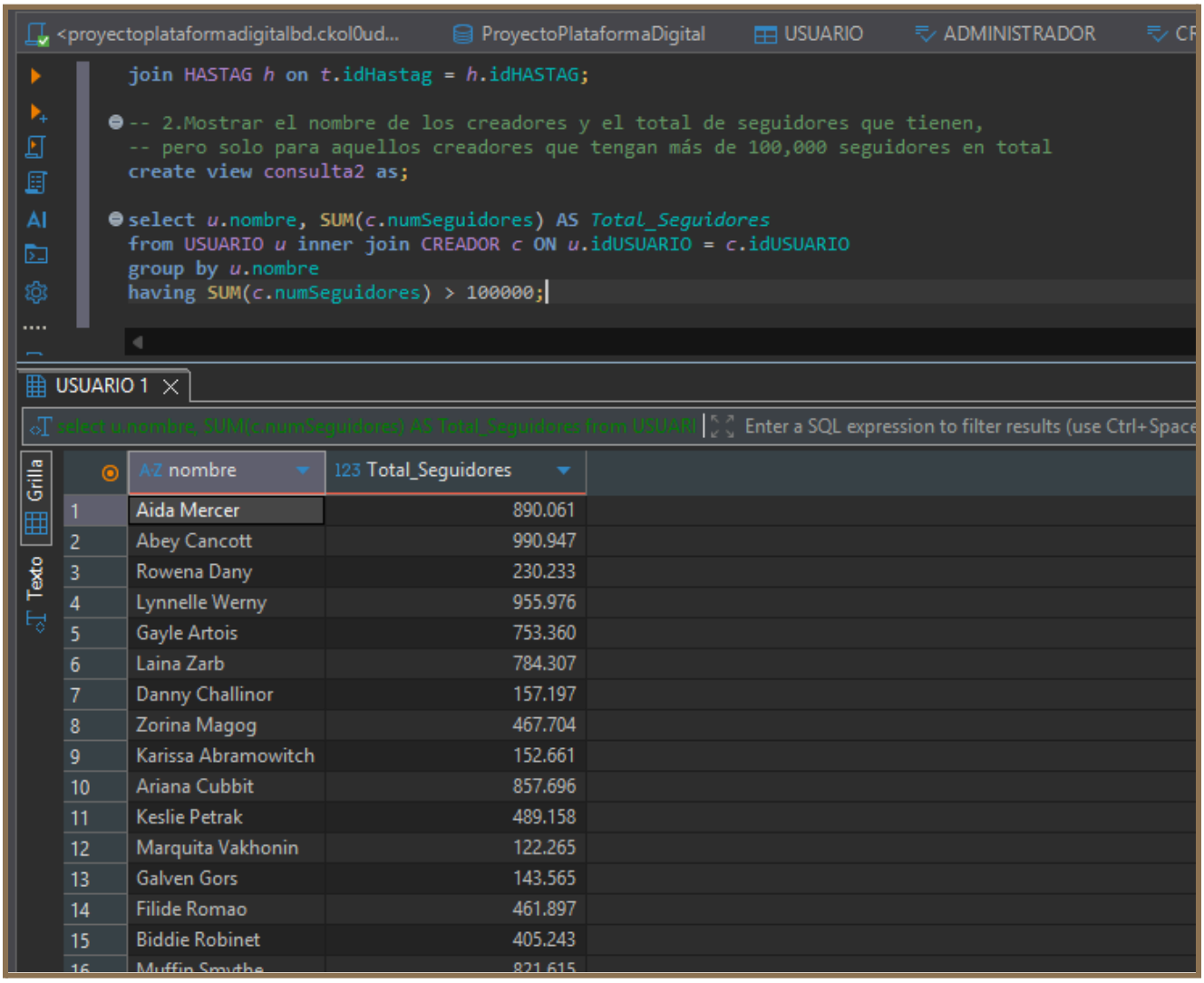


	Usuario_Comenta	titulo	Creador_Video	Hashtag
1	Mellicent Blumer	POV: eres mi gato	Loutitia Liggett	#beauty_2
2	Guillema Shone	Tutorial rapido	Joanie Bowld	#travel_5
3	Tandi Tredget	Storytime: lo que me paso	Ludwig Harkess	#travel_5
4	Ailis Snashall	Unboxing increible	Huntley Luxon	#travel_5
5	Sibyl Caillou	Review de producto	Loella Higgan	#lifestyle_6
6	Isidore Renish	Baile viral	Sharline Ferebee	#fyp_7
7	Bekki Sergeant	Baile viral	Sharline Ferebee	#fyp_7
8	Domini Rivelon	POV: eres mi gato	Shina Caddan	#fitness_8
9	Cissy Bowes	POV: eres mi gato	Shina Caddan	#fitness_8
10	Lora Lyes	POV: eres mi gato	Shina Caddan	#fitness_8
11	Maudie Gaitone	Baile viral	Stacy Grossier	#fitness_8
12	Elnar Kubiczek	Get ready with me	Fanny Kubista	#comedy_10
13	Prudy Jehan	Get ready with me	Fanny Kubista	#comedy_10
14	Chiquita O'Giany	Expectativa vs Realidad	Darrell Gosnay	#comedy_10
15	Care Chardin	Unboxing increible	Malorie Featonby	#funny_14
16	Sarette Brownsey	Vlog de domingo	Kristos Pavy	#tutorial_16
17	Jan Bygreaves	Expectativa vs Realidad	Sergei Kennicott	#fashion_19

CONSULTA Nº2

Enunciado: Mostrar el nombre de los creadores y el total de seguidores que tienen, pero solo para aquellos creadores que tengan más de 100,000 seguidores en total

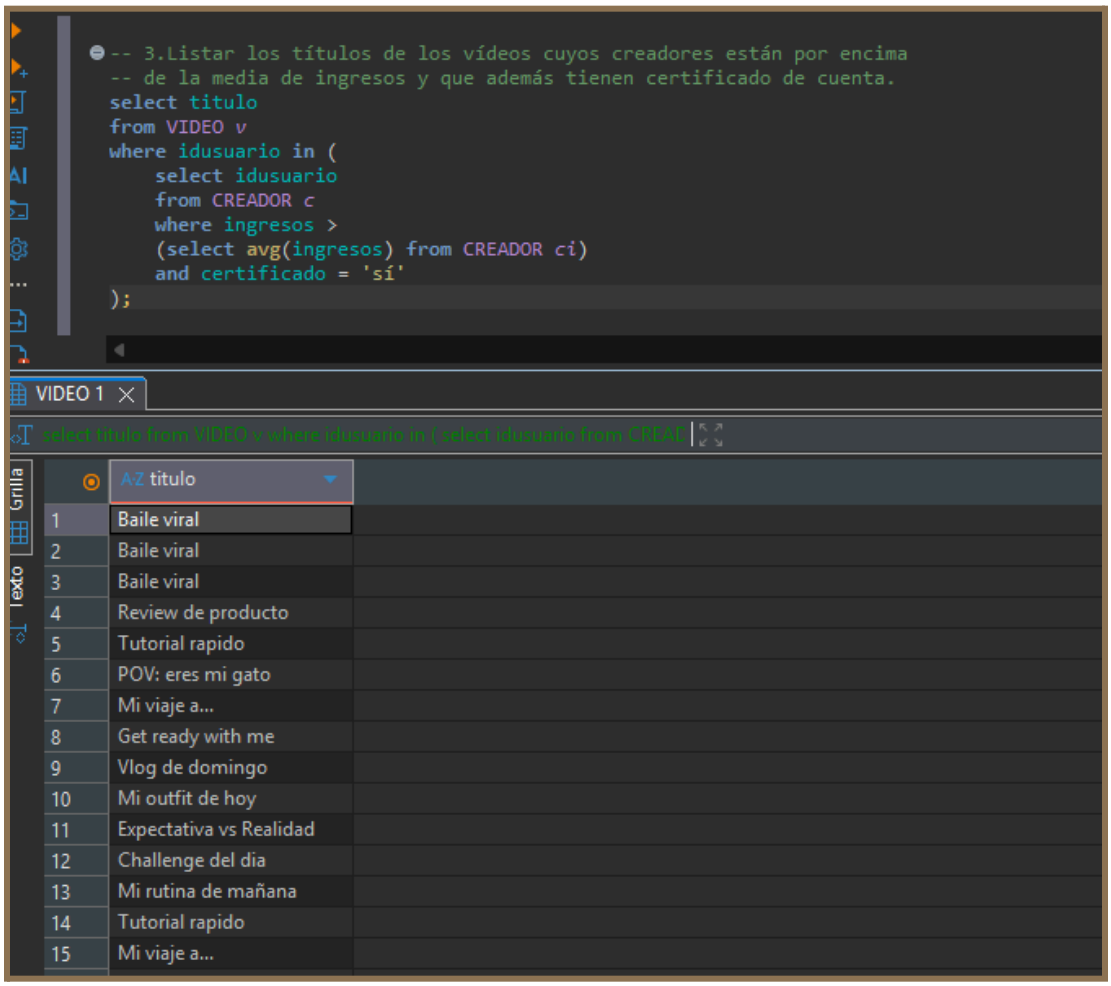
```
create view consulta2 as;
select u.nombre, SUM(c.numSeguidores) AS Total_Seguidores
from USUARIO u inner join CREADOR c ON u.idUSUARIO = c.idUSUARIO
group by u.nombre
having SUM(c.numSeguidores) > 100000;
```



CONSULTA Nº3

Enunciado: Listar los títulos de los vídeos cuyos creadores están por encima de la media de ingresos y que además tienen certificado de cuenta.

```
select titulo
from VIDEO v
where idusuario in (
  select idusuario
  from CREADOR c
  where ingresos >
  (select avg(ingresos) from CREADOR ci)
  and certificado = 'sí'
);
```



CONSULTA Nº4

Enunciado: Mostrar cuántos seguidores ha ganado cada creador por cada año. Debes mostrar el nombre del creador y el año de registro del usuario (para simular el crecimiento anual).

```
select
  u.nombre as creador,
  year(u.fecharegistro),
  sum(c.numseguidores) as total_seguidores
from USUARIO u
join CREADOR c on u.idusuario = c.idusuario
group by u.nombre, u.fechaRegistro ;
```

```
-- 4.Mostrar cuántos seguidores ha ganado cada creador por cada año. Debes mostrar el nombre
-- del creador y el año de registro del usuario (para simular el crecimiento anual).
select
    u.nombre as creador,
    year(u.fecharegistro),
    sum(c.numseguidores) as total_seguidores
from USUARIO u
join CREADOR c on u.idusuario = c.idusuario
group by u.nombre, u.fechaRegistro ;
```

RIO 1

u.nombre as creador, year(u.fecharegistro), sum(c.numseguidores) a | Enter a SQL expression to filter results (use Ctrl+Spa

creador	year(u.fecharegistro)	total_seguidores
Aida Mercer	2.021	890.061
Abey Cancott	2.020	990.947
Rowena Dany	2.021	230.233
Putnem Izachik	2.022	21.553
Lynnelle Werny	2.021	955.976
Gayle Artois	2.020	753.360
Laina Zarb	2.022	784.307
Danny Challinor	2.020	157.197
Zorina Magog	2.021	467.704
Karissa Abramowitch	2.022	152.661
Ariana Cubbit	2.022	857.696
Keslie Petrak	2.022	489.158
Clemmy Falkingham	2.021	20.847
Marquita Vakhonin	2.022	122.265
Galyen Gors	2.021	143.565

CONSULTA Nº5

Enunciado: Mostrar el nombre del usuario y el total de reacciones de tipo 'Me encanta' que ha realizado pero solo aquellos que han reaccionado a más de 3 videos distintos.

```
create view consulta5 as;
select
  u.nombre,
  u.email,
  count(r.idVIDEO) as total_reacciones_top
from USUARIO u
join REACCIONA r on u.idUSUARIO = r.idUSUARIOreacciona
where find_in_set('Me encanta', r.tipoReaccion) > 0
group by u.nombre, u.email
having count(r.idVIDEO) > 3;
```

```
-- 5.Mostrar el nombre del usuario y el total de reacciones de tipo 'Me encanta' que ha realizado,
-- pero solo para aquellos usuarios que han reaccionado a más de 3 vídeos distintos.
create view consulta5 as;

select
    u.nombre,
    u.email,
    count(r.idVIDEO) as total_reacciones_top
from USUARIO u
join REACCIONA r on u.idUSUARIO = r.idUSUARIOreacciona
where find_in_set('Me encanta', r.tipoReaccion) > 0
group by u.nombre, u.email
having count(r.idVIDEO) > 3;
```

JARIO 1

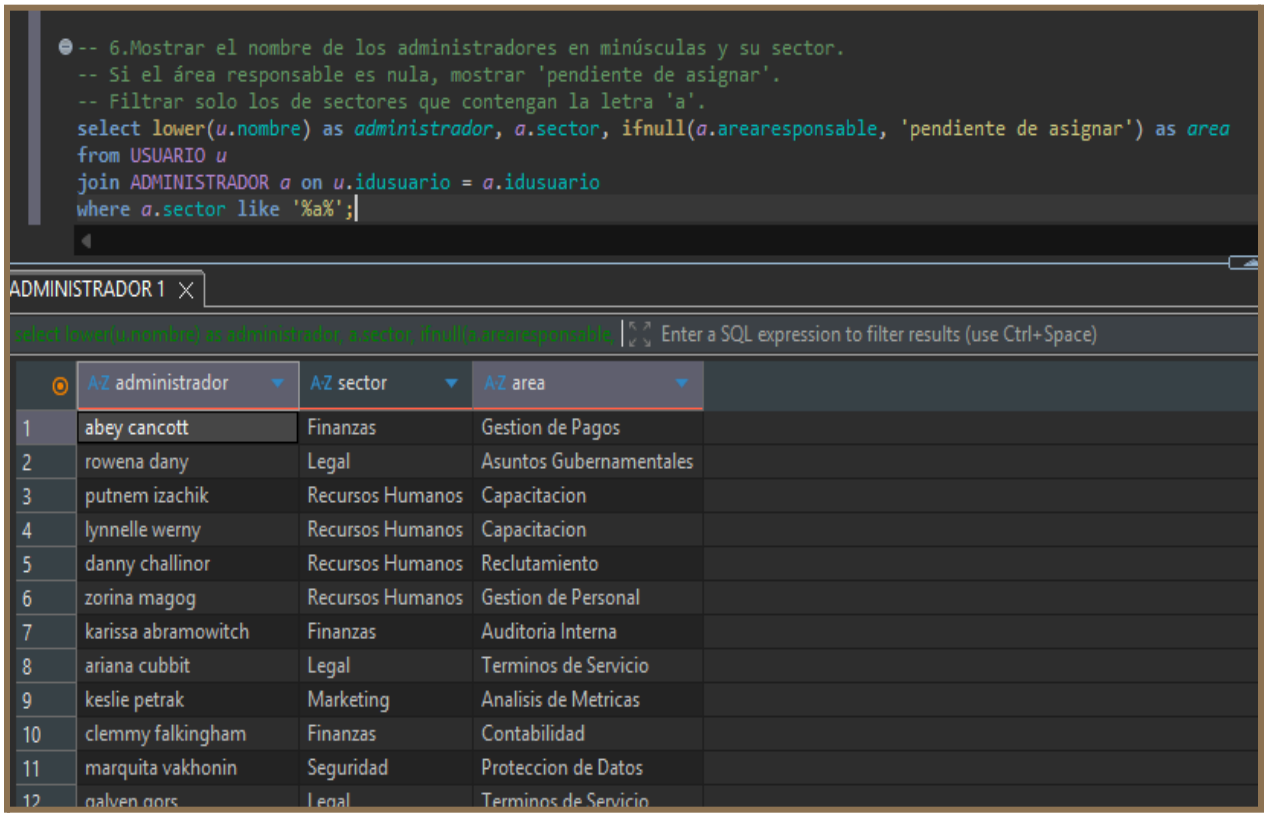
Enter a SQL expression to filter results (use Ctrl+Space)

nombre	email	total_reacciones_top
Brad Crowthe	bcrowthe9i@shop-pro.jp	4

CONSULTA Nº6

Enunciado: Mostrar el nombre de los administradores en minúsculas y su sector. Si el área responsable es nula, mostrar 'pendiente de asignar'.Filtrar solo los de sectores que contengan la letra 'a'.

```
select lower(u.nombre) as administrador, a.sector, ifnull(a.arearesponsable, 'pendiente de asignar')
as area
from USUARIO u
join ADMINISTRADOR a on u.idusuario = a.idusuario
where a.sector like '%a%';
```



	administrador	sector	area
1	abey cancott	Finanzas	Gestion de Pagos
2	rowena dany	Legal	Asuntos Gubernamentales
3	putnem izachik	Recursos Humanos	Capacitacion
4	lynnelle werny	Recursos Humanos	Capacitacion
5	danny challinor	Recursos Humanos	Reclutamiento
6	zorina magog	Recursos Humanos	Gestion de Personal
7	karissa abramowitch	Finanzas	Auditoria Interna
8	ariana cubbit	Legal	Terminos de Servicio
9	keslie petrak	Marketing	Analisis de Metricas
10	clemmy falkingham	Finanzas	Contabilidad
11	marquita vakhonin	Seguridad	Proteccion de Datos
12	galyen gors	Legal	Terminos de Servicio

CONSULTA Nº7

Enunciado: Mostrar el nombre del seguidor y el nombre del creador al que sigue, filtrando para que el usuario seguido sea estrictamente del tipo 'creador'.

```
select u.nombre as usuario_seguidor, u2.nombre as creador_seguido, u2.tipo
from SIGUE s
inner  join USUARIO u on s.idusuario = u.idusuario
inner join USUARIO u2 on s.idusuario_sigue = u2.idusuario
where u2.tipo = 'creador';
```

```
-- 7.Mostrar el nombre del seguidor y el nombre del creador al que sigue,
-- filtrando para que el usuario seguido sea estrictamente del tipo 'creador'.
select u.nombre as usuario_seguidor, u2.nombre as creador_seguido, u2.tipo
from SIGUE s
inner  join USUARIO u on s.idusuario = u.idusuario
inner join USUARIO u2 on s.idusuario_sigue = u2.idusuario
where u2.tipo = 'creador';
```

USUARIO 1

select u.nombre as usuario_seguidor, u2.nombre as creador_seguido, u2.tip | Enter a SQL expression to filter resu

	usuario_seguidor	creador_seguido	tipo
1	Ramona Passman	Rowena Dany	creador
2	Clareta Elliott	Danny Challinor	creador
3	Prudy Jehan	Karissa Abramowitch	creador
4	Zed Palfreman	Karissa Abramowitch	creador
5	Burt Proppers	Karissa Abramowitch	creador
6	Adriena Wakerley	Ariana Cubbit	creador
7	Noell Southway	Filide Romao	creador
8	Barnabe Castillo	Sunny Panks	creador
9	Roxanne Bukac	Martynne Symms	creador
10	Reena Isacq	Martynne Symms	creador
11	Dante Ogles	Martynne Symms	creador
12	Harli Thebeau	Felita Ledingham	creador
13	Sada Bristowe	Iolande Brunini	creador
14	Claudianus Ferrao	Tadd Khomishin	creador
15	Andrey Bentley	Tadd Khomishin	creador

6.FUNCIONES Y PROCEDIMIENTOS DEL PROYECTO.

Función 1:

- **Enunciado:** Crear una función que, dado un número de seguidores, devuelva la categoría del creador ('Estrella' para más de 500.000, 'Popular' para más de 100.000, o 'En crecimiento' para el resto).
- **Código SQL:**

```
DELIMITER //
CREATE FUNCTION categoria_creador(idCreador INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
  DECLARE categoria VARCHAR(20);
        DECLARE numeroSeguidores int;

        SELECT c.numSeguidores
        into numeroSeguidores
        from CREADOR c
        where c.idUSUARIO  =idCreador;

        IF numeroSeguidores >= 500000 THEN
    SET categoria = 'Estrella';
  ELSEIF numeroSeguidores between 100000 and 500000 THEN
    SET categoria = 'Popular';
  ELSE
    SET categoria = 'En crecimiento';
  END IF;
  RETURN categoria;
END //
DELIMITER ;
SELECT categoria_creador(150000) AS Categoria;
```

```sql
-- 1 Crear una función que, dado un número de seguidores,
--  devuelva la categoría del creador ('Estrella' para más de 500.000,
-- 'Popular' para más de 100.000, o 'En crecimiento' para el resto).
DELIMITER //
CREATE FUNCTION categoria_creador(idCreador INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE categoria VARCHAR(20);
    DECLARE numeroSeguidores int;

    SELECT c.numSeguidores
    into numeroSeguidores
    from CREADOR c
    where c.idUSUARIO  =idCreador;

    IF numeroSeguidores >= 500000 THEN
        SET categoria = 'Estrella';
    ELSEIF numeroSeguidores between 100000 and 500000 THEN
        SET categoria = 'Popular';
    ELSE
        SET categoria = 'En crecimiento';
    END IF;
    RETURN categoria;
END //
DELIMITER ;
SELECT categoria_creador(150000) AS Categoria;
```

Resultados 1

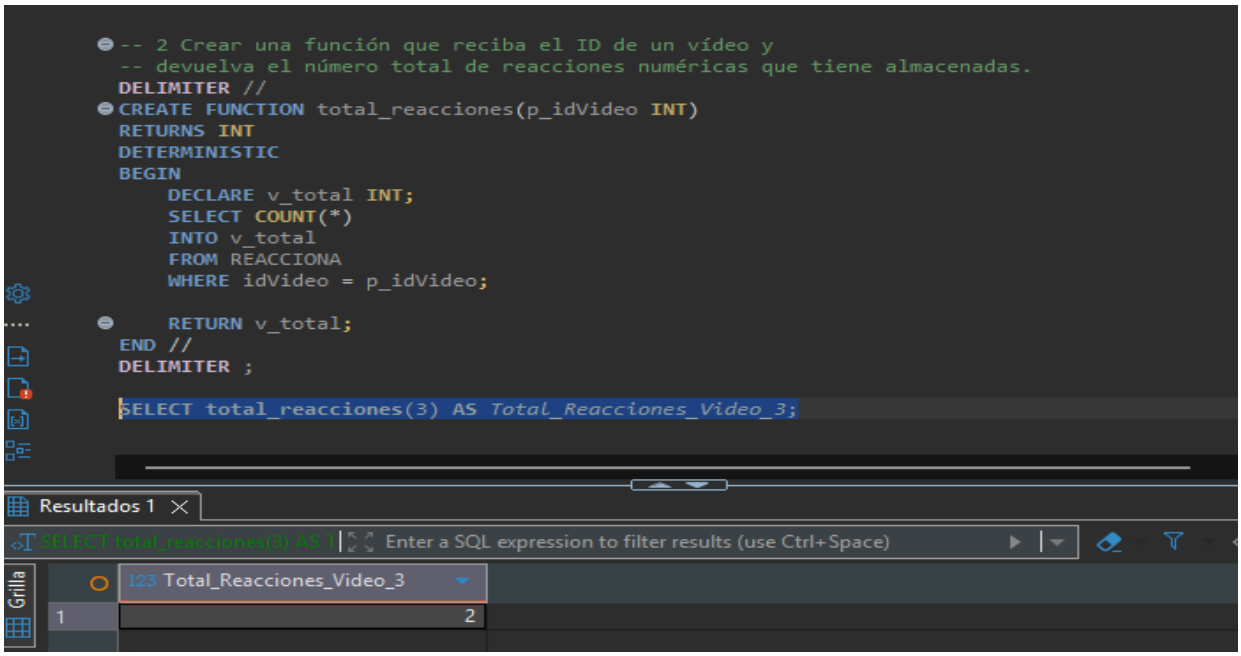
SELECT categoria_creador(1500 | Enter a SQL expression to filter results (use Ctrl+Space)

	Categoria
1	En crecimiento

Función 2:

- **Enunciado:** Crear una función que reciba el ID de un vídeo y devuelva el número total de reacciones numéricas que tiene almacenadas.
- **Código SQL:**

```
DELIMITER //
CREATE FUNCTION total_reacciones(p_idVideo INT)
RETURNS INT
DETERMINISTIC
BEGIN
  DECLARE v_total INT;
  SELECT COUNT(*)
        INTO v_total
        FROM REACCIONA
        WHERE idVideo = p_idVideo;
  RETURN v_total;
END //
DELIMITER ;
SELECT total_reacciones(3) AS Total_Reacciones_Video_3;
```



Procedimiento 1:

- **Enunciado:** Crear un procedimiento que reciba el ID de un usuario creador y muestre su nombre, cantidad de seguidores y su rango en la plataforma haciendo uso de la función fn_categoria_creador.
- **Código SQL:**

```
drop procedure if exists info_creador;
DELIMITER //
CREATE PROCEDURE info_creador(IN p_idUsuario INT)
BEGIN
        declare nombre varchar(50);
        declare seguidores int;
        declare rango varchar(20);
 SELECT
   u.nombre,
   c.numSeguidores,
   categoria_creador(c.numSeguidores) AS Rango_Plataforma
 into nombre,seguidores,rango
 FROM USUARIO u
 INNER JOIN CREADOR c ON u.idUSUARIO = c.idUSUARIO
 WHERE u.idUSUARIO = p_idUsuario;
        select nombre,seguidores,rango;
END //
DELIMITER ;
CALL info_creador(2);
```

```sql
-- 1. Crear un procedimiento que reciba el ID de un usuario creador
-- y muestre su nombre, cantidad de seguidores y
-- su rango en la plataforma haciendo uso de la función fn_categoria_creador.
drop procedure if exists info_creador;
DELIMITER //
CREATE PROCEDURE info_creador(IN p_idUsuario INT)
BEGIN
    declare nombre varchar(50);
    declare seguidores int;
    declare rango varchar(20);
   SELECT
       u.nombre,
       c.numSeguidores,
       categoria_creador(c.numSeguidores) AS Rango_Plataforma
   into nombre,seguidores,rango
   FROM USUARIO u
   INNER JOIN CREADOR c ON u.idUSUARIO = c.idUSUARIO
   WHERE u.idUSUARIO = p_idUsuario;
    select nombre,seguidores,rango;
END //
DELIMITER ;
CALL info_creador(2);
```

ltados 1 | Estadísticas 1

L info_creador(2) | Enter a SQL expression to filter results (use Ctrl+Space)

A-Z nombre	123 seguidores	A-Z rango
Sigismondo Androlli	990.948	En crecimiento

Procedimiento 2:

- **Enunciado:** Crear un procedimiento que reciba el ID de un creador y actualice su estado de certificado a 'Si', pero solo si tiene estrictamente más de 100.000 seguidores.
- **Código SQL:**

```
DELIMITER //
CREATE PROCEDURE sp_certificar_cuenta(IN p_idUsuario INT)
BEGIN
        if exists (select 1 from CREADOR c where c.IdUsuario=p_idUsuario) then
          UPDATE CREADOR
          SET certificado = 'Si'
          WHERE idUSUARIO = p_idUsuario AND numSeguidores > 100000;
        else
                signal sqlstate '45000'
                set message_text='No existe el creador';
        end if;
END //
DELIMITER ;
CALL sp_certificar_cuenta(3);
SELECT idUSUARIO, numSeguidores, certificado FROM CREADOR WHERE idUSUARIO = 3;
```

```
-- 2 Crear un procedimiento que reciba el ID de un creador y actualice su estado de certificado
-- a 'Si', pero solo si tiene estrictamente más de 100.000 seguidores
DELIMITER //
CREATE PROCEDURE sp_certificar_cuenta(IN p_idUsuario INT)
BEGIN
    if exists (select 1 from CREADOR c where c.IdUsuario=p_idUsuario) then
        UPDATE CREADOR
        SET certificado = 'Si'
        WHERE idUSUARIO = p_idUsuario AND numSeguidores > 100000;
    else
        signal sqlstate '45000'
        set message_text='No existe el creador';
    end if;
END //
DELIMITER ;
CALL sp_certificar_cuenta(3);

SELECT idUSUARIO, numSeguidores, certificado FROM CREADOR WHERE idUSUARIO = 3;
```

OR 1

idUSUARIO, numSeguid | Enter a SQL expression to filter results (use Ctrl+Space)

idUSUARIO	numSeguidores	certificado
3	230.233	Si

```
-- 2 Crear un procedimiento que reciba el ID de un creador y actualice su estado de certificado
-- a 'Si', pero solo si tiene estrictamente más de 100.000 seguidores
DELIMITER //
CREATE PROCEDURE sp_certificar_cuenta(IN p_idUsuario INT)
BEGIN
    if exists (select 1 from CREADOR c where c.IdUsuario=p_idUsuario) then
        UPDATE CREADOR
        SET certificado = 'Si'
        WHERE idUSUARIO = p_idUsuario AND numSeguidores > 100000;
    else
        signal sqlstate '45000'
        set message_text='No existe el creador';
    end if;
END //
DELIMITER ;
CALL sp_certificar_cuenta(3);

SELECT idUSUARIO, numSeguidores, certificado FROM CREADOR WHERE idUSUARIO = 3;
CALL sp_certificar_cuenta(9999);
```

CREADOR 1

CALL sp_certificar_cuenta(9999) | Enter a SQL expression to filter results (use Ctrl+Space)

```
SQL Error [1644] [45000]: No existe el creador
```

```
CALL sp_certificar_cuenta(9999)
```

Procedimiento 3:

- **Enunciado:** Crear un procedimiento que permita actualizar rápidamente el área responsable de un administrador pasándole su ID y el nombre del nuevo departamento.
- **Código SQL:**

```
DELIMITER //
CREATE PROCEDURE sp_cambiar_area_admin(IN p_idUsuario INT, IN p_nuevaArea
VARCHAR(100))
BEGIN
  UPDATE ADMINISTRADOR
  SET areaResponsable = p_nuevaArea
  WHERE idUSUARIO = p_idUsuario;
END //
DELIMITER ;
CALL sp_cambiar_area_admin(1, 'Seguridad Global');
SELECT idUSUARIO, areaResponsable FROM ADMINISTRADOR WHERE idUSUARIO = 1;
```

```
-- 3 Crear un procedimiento que permita actualizar rápidamente el área responsable de un
-- administrador pasándole su ID y el nombre del nuevo departamento.
DELIMITER //
CREATE PROCEDURE sp_cambiar_area_admin(IN p_idUsuario INT, IN p_nuevaArea VARCHAR(100))
BEGIN
    UPDATE ADMINISTRADOR
    SET areaResponsable = p_nuevaArea
    WHERE idUSUARIO = p_idUsuario;
END //
DELIMITER ;

CALL sp_cambiar_area_admin(1, 'Seguridad Global');

SELECT idUSUARIO, areaResponsable FROM ADMINISTRADOR WHERE idUSUARIO = 1;
```

STRADOR 1

idUSUARIO, areaRespor | Enter a SQL expression to filter results (use Ctrl+Space)

123 idUSUARIO	A-Z areaResponsable
1	Seguridad Global

7. TRIGGERS DEL PROYECTO

Trigger 1:

- **Enunciado:** Crear un trigger que se active cada vez que un usuario comience a seguir a otro (es decir, al insertar un nuevo registro en la tabla SIGUE). El trigger debe aumentar automáticamente en 1 el contador de seguidores (numSeguidores) del creador que ha recibido el seguimiento.
- **Código SQL:**

```
DELIMITER //
CREATE TRIGGER nuevo_seguidor
AFTER INSERT ON SIGUE
FOR EACH ROW
BEGIN
 UPDATE CREADOR
 SET numSeguidores = numSeguidores + 1
 WHERE idUSUARIO = NEW.idUSUARIO_sigue;
END //
DELIMITER ;
SELECT idUSUARIO, numSeguidores FROM CREADOR WHERE idUSUARIO = 2;
INSERT INTO SIGUE (idUSUARIO, idUSUARIO_sigue) VALUES (50, 2);
SELECT idUSUARIO, numSeguidores FROM CREADOR WHERE idUSUARIO = 2;
```

```sql
-- 2 Crear un trigger que se active cada vez que un usuario comience a seguir a otro
-- (es decir, al insertar un nuevo registro en la tabla SIGUE).
-- El trigger debe aumentar automáticamente en 1 el contador de seguidores (numSeguidores)
-- del creador que ha recibido el seguimiento.
DELIMITER //
CREATE TRIGGER nuevo_seguidor
AFTER INSERT ON SIGUE
FOR EACH ROW
BEGIN
   UPDATE CREADOR
   SET numSeguidores = numSeguidores + 1
   WHERE idUSUARIO = NEW.idUSUARIO_sigue;
END //
DELIMITER ;

SELECT idUSUARIO, numSeguidores FROM CREADOR WHERE idUSUARIO = 2;

INSERT INTO SIGUE (idUSUARIO, idUSUARIO_sigue) VALUES (50, 2);

SELECT idUSUARIO, numSeguidores FROM CREADOR WHERE idUSUARIO = 2;
```

Estadísticas 1

Name	Value
Queries	3
Updated Rows	0
Execute time	0.2s
Fetch time	0.0s
Total time	0.2s
Start time	2026-03-23 08:43:15.011
Finish time	2026-03-23 08:43:15.259

```sql
-- 2 Crear un trigger que se active cada vez que un usuario comience a seguir a otro
-- (es decir, al insertar un nuevo registro en la tabla SIGUE).
-- El trigger debe aumentar automáticamente en 1 el contador de seguidores (numSeguidores)
-- del creador que ha recibido el seguimiento.
DELIMITER //
CREATE TRIGGER nuevo_seguidor
AFTER INSERT ON SIGUE
FOR EACH ROW
BEGIN
   UPDATE CREADOR
   SET numSeguidores = numSeguidores + 1
   WHERE idUSUARIO = NEW.idUSUARIO_sigue;
END //
DELIMITER ;

SELECT idUSUARIO, numSeguidores FROM CREADOR WHERE idUSUARIO = 2;

INSERT INTO SIGUE (idUSUARIO, idUSUARIO_sigue) VALUES (50, 2);

SELECT idUSUARIO, numSeguidores FROM CREADOR WHERE idUSUARIO = 2;
```

READOR 1

Enter a SQL expression to filter results (use Ctrl+Space)

	idUSUARIO	numSeguidores
1	2	990.947

```
-- 2 Crear un trigger que se active cada vez que un usuario comience a seguir a otro
-- (es decir, al insertar un nuevo registro en la tabla SIGUE).
-- El trigger debe aumentar automáticamente en 1 el contador de seguidores (numSeguidores)
-- del creador que ha recibido el seguimiento.
DELIMITER //
CREATE TRIGGER nuevo_seguidor
AFTER INSERT ON SIGUE
FOR EACH ROW
BEGIN
   UPDATE CREADOR
   SET numSeguidores = numSeguidores + 1
   WHERE idUSUARIO = NEW.idUSUARIO_sigue;
END //
DELIMITER ;

SELECT idUSUARIO, numSeguidores FROM CREADOR WHERE idUSUARIO = 2;

INSERT INTO SIGUE (idUSUARIO, idUSUARIO_sigue) VALUES (50, 2);

SELECT idUSUARIO, numSeguidores FROM CREADOR WHERE idUSUARIO = 2;
```

Estadísticas 1

Name	Value
Updated Rows	1
Execute time	0.2s
Start time	Mon Mar 23 08:44:11 CET 2026
Finish time	Mon Mar 23 08:44:12 CET 2026
Query	INSERT INTO SIGUE (idUSUARIO, idUSUARIO_sigue) VALUES (50, 2)


```
-- 2 Crear un trigger que se active cada vez que un usuario comience a seguir a otro
-- (es decir, al insertar un nuevo registro en la tabla SIGUE).
-- El trigger debe aumentar automáticamente en 1 el contador de seguidores (numSeguidores)
-- del creador que ha recibido el seguimiento.
DELIMITER //
CREATE TRIGGER nuevo_seguidor
AFTER INSERT ON SIGUE
FOR EACH ROW
BEGIN
   UPDATE CREADOR
   SET numSeguidores = numSeguidores + 1
   WHERE idUSUARIO = NEW.idUSUARIO_sigue;
END //
DELIMITER ;

SELECT idUSUARIO, numSeguidores FROM CREADOR WHERE idUSUARIO = 2;

INSERT INTO SIGUE (idUSUARIO, idUSUARIO_sigue) VALUES (50, 2);

SELECT idUSUARIO, numSeguidores FROM CREADOR WHERE idUSUARIO = 2;
```


CREADOR 1

SELECT idUSUARIO, numSeguid | Enter a SQL expression to filter results (use Ctrl+Space)

Grilla

	123 idUSUARIO	123 numSeguidores
1	2	990.948

Trigger 2:

- **Enunciado:** Crear un trigger que salte cada vez que un usuario reaccione a un vídeo (al insertar en la tabla REACCIONA). Si la reacción es positiva ('Like' o 'Me encanta'), el trigger debe buscar al creador dueño de ese vídeo y sumarle 1 a su contador de likes totales (numLikes).
- **Código SQL:**

```
DELIMITER //
CREATE TRIGGER reaccion_positiva
AFTER INSERT ON REACCIONA
FOR EACH ROW
BEGIN
  DECLARE idCreador INT;
  -- Si la reacción es positiva, actuamos:
  IF NEW.tipoReaccion IN ('Like', 'Me encanta') THEN
    -- 1. Buscamos de quién es el vídeo reaccionado
    SELECT idUSUARIO INTO idCreador FROM VIDEO WHERE idVIDEO = NEW.idVideo;

    -- 2. Le sumamos 1 a los likes totales de ese creador
    UPDATE CREADOR
    SET numLikes = numLikes + 1
    WHERE idUSUARIO = idCreador;
  END IF;
END //
DELIMITER ;
SELECT idUSUARIO, numLikes FROM CREADOR WHERE idUSUARIO = 2;
INSERT INTO REACCIONA (idVideo, idUsuarioReacciona, tipoReaccion, fechaReaccion)
VALUES (10, 5, 'Like', '2024-05-10');
SELECT idUSUARIO, numLikes FROM CREADOR WHERE idUSUARIO = 2;
```

```
-- TRIGGER

-- Crear un trigger que salte cada vez que un usuario reaccione a un vídeo
-- (al insertar en la tabla REACCIONA). Si la reacción es positiva ('Like' o 'Me encanta'),
-- el trigger debe buscar al creador dueño de ese vídeo
-- y sumarle 1 a su contador de likes totales (numLikes).
DELIMITER //
CREATE TRIGGER reaccion_positiva
AFTER INSERT ON REACCIONA
FOR EACH ROW
BEGIN
   DECLARE idCreador INT;

   -- Si la reacción es positiva, actuamos:
   IF NEW.tipoReaccion IN ('Like', 'Me encanta') THEN
       -- 1. Buscamos de quién es el vídeo reaccionado
       SELECT idUSUARIO INTO idCreador FROM VIDEO WHERE idVIDEO = NEW.idVideo;

       -- 2. Le sumamos 1 a los likes totales de ese creador
       UPDATE CREADOR
       SET numLikes = numLikes + 1
       WHERE idUSUARIO = idCreador;
   END IF;
END //
DELIMITER ;

SELECT idUSUARIO, numLikes FROM CREADOR WHERE idUSUARIO = 2;

INSERT INTO REACCIONA (idVideo, idUsuarioReacciona, tipoReaccion, fechaReaccion)
VALUES (10, 5, 'Like', '2024-05-10');

SELECT idUSUARIO, numLikes FROM CREADOR WHERE idUSUARIO = 2;


-- 2 Crear un trigger que se active cada vez que un usuario comience a seguir a otro
-- (es decir, al insertar un nuevo registro en la tabla SIGUE).
-- El trigger debe aumentar automáticamente en 1 el contador de seguidores (numSeguidores)
-- del creador que ha recibido el seguimiento.
```

Estadísticas 1

Name	Value
Queries	3
Updated Rows	0
Execute time	0.2s
Fetch time	0.0s
Total time	0.2s
Start time	2026-03-23 08:30:37.414
Finish time	2026-03-23 08:30:37.834

```sql
-- Crear un trigger que salte cada vez que un usuario reaccione a un vídeo
-- (al insertar en la tabla REACCIONA). Si la reacción es positiva ('Like' o 'Me encanta'),
-- el trigger debe buscar al creador dueño de ese vídeo
-- y sumarle 1 a su contador de likes totales (numLikes).
DELIMITER //
CREATE TRIGGER reaccion_positiva
AFTER INSERT ON REACCIONA
FOR EACH ROW
BEGIN
   DECLARE idCreador INT;

   -- Si la reacción es positiva, actuamos:
   IF NEW.tipoReaccion IN ('Like', 'Me encanta') THEN
       -- 1. Buscamos de quién es el vídeo reaccionado
       SELECT idUSUARIO INTO idCreador FROM VIDEO WHERE idVIDEO = NEW.idVideo;

       -- 2. Le sumamos 1 a los likes totales de ese creador
       UPDATE CREADOR
       SET numLikes = numLikes + 1
       WHERE idUSUARIO = idCreador;
   END IF;
END //
DELIMITER ;

SELECT idUSUARIO, numLikes FROM CREADOR WHERE idUSUARIO = 2;

INSERT INTO REACCIONA (idVideo, idUsuarioReacciona, tipoReaccion, fechaReaccion)
VALUES (10, 5, 'Like', '2024-05-10');

SELECT idUSUARIO, numLikes FROM CREADOR WHERE idUSUARIO = 2;


-- 2 Crear un trigger que se active cada vez que un usuario comience a seguir a otro
-- (es decir, al insertar un nuevo registro en la tabla SIGUE).
```

CREADOR 1

SELECT idUSUARIO, numLikes

	idUSUARIO	numLikes
1	2	21645818

```sql
-- Crear un trigger que salte cada vez que un usuario reaccione a un vídeo
-- (al insertar en la tabla REACCIONA). Si la reacción es positiva ('Like' o 'Me encanta'),
-- el trigger debe buscar al creador dueño de ese vídeo
-- y sumarle 1 a su contador de likes totales (numLikes).
DELIMITER //
CREATE TRIGGER reaccion_positiva
AFTER INSERT ON REACCIONA
FOR EACH ROW
BEGIN
   DECLARE idCreador INT;

   -- Si la reacción es positiva, actuamos:
   IF NEW.tipoReaccion IN ('Like', 'Me encanta') THEN
       -- 1. Buscamos de quién es el vídeo reaccionado
       SELECT idUSUARIO INTO idCreador FROM VIDEO WHERE idVIDEO = NEW.idVideo;

       -- 2. Le sumamos 1 a los likes totales de ese creador
       UPDATE CREADOR
       SET numLikes = numLikes + 1
       WHERE idUSUARIO = idCreador;
   END IF;
END //
DELIMITER ;

SELECT idUSUARIO, numLikes FROM CREADOR WHERE idUSUARIO = 2;

INSERT INTO REACCIONA (idVideo, idUsuarioReacciona, tipoReaccion, fechaReaccion)
VALUES (10, 5, 'Like', '2024-05-10');

SELECT idUSUARIO, numLikes FROM CREADOR WHERE idUSUARIO = 2;


-- 2 Crear un trigger que se active cada vez que un usuario comience a seguir a otro
-- (es decir, al insertar un nuevo registro en la tabla SIGUE).
```

Estadísticas 1

Name	Value
Updated Rows	1
Execute time	0.2s
Start time	Mon Mar 23 08:41:46 CET 2026
Finish time	Mon Mar 23 08:41:46 CET 2026
Query	INSERT INTO REACCIONA (idVideo, idUsuarioReacciona, tipoReaccion, fechaReaccion) VALUES (10, 5, 'Like', '2024-05-10')

```sql
-- Crear un trigger que salte cada vez que un usuario reaccione a un vídeo
-- (al insertar en la tabla REACCIONA). Si la reacción es positiva ('Like' o 'Me encanta'),
-- el trigger debe buscar al creador dueño de ese vídeo
-- y sumarle 1 a su contador de likes totales (numLikes).
DELIMITER //
CREATE TRIGGER reaccion_positiva
AFTER INSERT ON REACCIONA
FOR EACH ROW
BEGIN
   DECLARE idCreador INT;

   -- Si la reacción es positiva, actuamos:
   IF NEW.tipoReaccion IN ('Like', 'Me encanta') THEN
       -- 1. Buscamos de quién es el vídeo reaccionado
       SELECT idUSUARIO INTO idCreador FROM VIDEO WHERE idVIDEO = NEW.idVideo;

       -- 2. Le sumamos 1 a los likes totales de ese creador
       UPDATE CREADOR
       SET numLikes = numLikes + 1
       WHERE idUSUARIO = idCreador;
   END IF;
END //
DELIMITER ;

SELECT idUSUARIO, numLikes FROM CREADOR WHERE idUSUARIO = 2;

INSERT INTO REACCIONA (idVideo, idUsuarioReacciona, tipoReaccion, fechaReaccion)
VALUES (10, 5, 'Like', '2024-05-10');

SELECT idUSUARIO, numLikes FROM CREADOR WHERE idUSUARIO = 2;


-- 2 Crear un trigger que se active cada vez que un usuario comience a seguir a otro
-- (es decir, al insertar un nuevo registro en la tabla SIGUE).
```

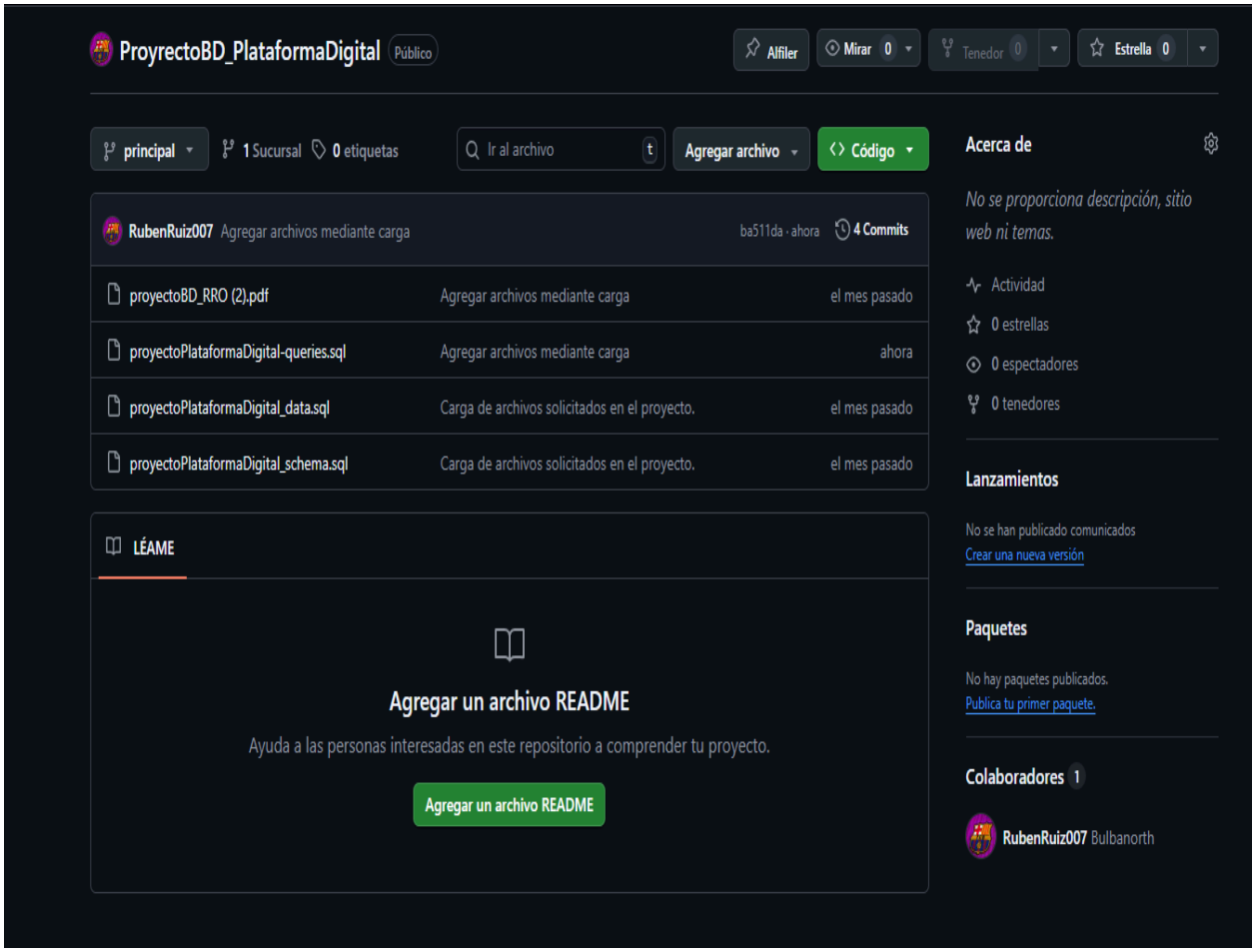
EADOR 1

ECT idUSUARIO, numLikes | Enter a SQL expression to filter results (use Ctrl+Space)

idUSUARIO	numLikes
2	21645818

8.SUBIDA A GITHUB

Como parte del trabajo se nos pedía un fichero sql con el esquema (miproyecto-schema.sql). Otro fichero sql con los datos (miproyecto-data.sql) y un último fichero con las consultas, vistas, funciones, procedimientos y triggers que has realizado(miproyecto-queries.sql)



9.CONCLUSIÓN DEL PROYECTO

Con la realización de este proyecto he aprendido a organizar la información de manera más clara y útil, como se haría en una aplicación web real. Me he dado cuenta de lo importante que es pensar bien el diseño de la base de datos antes de empezar a programar. Además, me ha servido para poner en práctica lo que hemos visto en clase, usando un ejemplo cercano como una plataforma de vídeos, y también para mejorar la forma en que estructura un proyecto de principio a fin.

En la siguiente actividad, tuve que realizar la carga de datos para mi proyecto, lo que me ayudó a profundizar a la hora de manejar el DBeaver, el control de las unidades en el sql, ya que muchas veces me daba error a la hora de añadir datos porque se me quedaba corto e incluso el empleo de DDL y DML para borrar datos de una columna por si me equivocaba a la hora de importar.Por cosas a mejorar, podría ser la complejidad de las consultas ya que no tengo un nivel tan alto como para complicarme a la hora de hacerlas.

Para la última parte del trabajo tuve en su mayoría problemas relacionados con la realización de una nueva base de datos, ya que se me acabó el saldo en AWS. Por lo tanto, tuve que hacer de nuevo todo el trabajo lo cual no me pareció complicado porque ya estaba hecho y solo debía de copiar, aunque si gastó bastante tiempo. Por otra parte, mejoré la presentación del trabajo como nos recomendó la profesora la última vez. Por último, realicé consultas de acuerdo al nivel que tenía en ese momento.

Enlace a GitHub :

https://github.com/RubenRuiz007/ProyrectoBD_PlataformaDigital

Puerto de enlace :

proyectoplataformaaudioweb.cpwgoimys28c.us-east-1.rds.amazonaws.com